MINI-PROJECT REPORT

VEHICLE RENTAL SYSTEM USING INHERITANCE

Object-Oriented Programming using C++

Submitted by	Tiya
Roll Number	2556508
Class / Section	B.Tech CSE – B
Semester	3rd Semester
University	CGC University

1. Introduction

This mini-project, titled **Vehicle Rental System Using Inheritance**, is developed in C++ to demonstrate object-oriented programming concepts. The system allows a user to select a car or a bike, enter vehicle details and rental duration, and view the calculated rental charges.

The project uses a common base class for shared vehicle information and two derived classes for vehicle-specific rental calculations.

2. Objectives

• To understand classes and objects in C++.
• To implement single inheritance using a base class and derived classes.
• To use constructors to initialize vehicle information.
• To apply encapsulation by keeping data members private.
• To demonstrate function overriding and runtime polymorphism.
• To calculate rental charges according to vehicle type.
• To validate user input, especially the rental duration.

3. Problem Statement

A vehicle rental service needs a simple console-based system that can calculate rental charges for different vehicle types. Cars and bikes share common information, such as vehicle ID, model, rental duration and daily rate, but their charge calculations may differ. The program should display the rental details and handle invalid menu choices or non-positive rental durations.

4. Project Description and Features

The program displays a menu with three options: Rent a Car, Rent a Bike and Exit. After the user selects a vehicle type, the program asks for the vehicle ID, model and number of rental days. It then creates the appropriate object, displays the vehicle information and calculates the total rental charge.

Main features:
• Menu-driven console interface.
• Separate Car and Bike classes derived from Vehicle.
• Different rental-charge calculations for cars and bikes.
• Display of vehicle ID, model, rental days, daily rate and total charge.
• Validation of menu input and rental duration.

5. Software and Hardware Requirements

Software: C++ compiler (GCC/G++), MSYS2 UCRT64 or another C++ IDE/compiler, and Windows/Linux operating system.
Hardware: A computer or laptop with a keyboard, display and sufficient memory to run a C++ compiler.

6. Object-Oriented Programming Concepts Used

Classes and Objects: Vehicle, Car and Bike are classes. Objects of Car or Bike are created when the user selects a vehicle type.

Single Inheritance: Car and Bike inherit the common properties and functions of the Vehicle base class. Each derived class has one direct base class.

Encapsulation: The data members vehicleID, model, rentalDays and dailyRate are declared private in Vehicle. They are accessed through controlled member functions.

Constructors: The Vehicle constructor initializes the common vehicle information. The Car and Bike constructors call the base-class constructor and provide their respective daily rates.

Function Overriding: Car and Bike provide their own versions of calculateRentalCharge() and displayDetails().

Runtime Polymorphism: A Vehicle pointer refers to a Car or Bike object. Virtual functions ensure that the derived-class implementation is called at runtime.

7. Class Design

Vehicle (Base Class)
Stores vehicle ID, model, rental days and daily rate. It declares calculateRentalCharge() as a pure virtual function and provides a common displayDetails() function.

Car (Derived Class)
Inherits Vehicle. It uses a daily rate of Rs. 1,500 and adds a 10% service charge to the base rental amount.

Bike (Derived Class)
Inherits Vehicle. It uses a daily rate of Rs. 500 and calculates the charge by multiplying the daily rate by the number of rental days.

8. Class Relationship

```
Vehicle
/ \\
Car Bike
```

Car and Bike both inherit directly from Vehicle. This is an example of single inheritance for each derived class.

9. Rental Charge Calculation

Car: Base amount = Daily rate × Rental days. Total = Base amount + 10% service charge.
Example: Rs. 1,500 × 2 = Rs. 3,000. Service charge = Rs. 300. Total = **Rs. 3,300**.

Bike: Total = Daily rate × Rental days.
Example: Rs. 500 × 3 = **Rs. 1,500**.

Note: These rates and the car service charge are assumptions selected for this mini-project. They are not fixed values specified in the assignment.

10. Algorithm / Working Procedure

1. Start the program.
2. Display the main menu.
3. Read the user's choice.
4. If the choice is Exit, display a thank-you message and stop.
5. If the choice is invalid, display an error message and show the menu again.
6. Ask for the vehicle ID and model.
7. Read the rental duration and check that it is a valid positive number.
8. Create a Car object or Bike object according to the selected option.
9. Use the Vehicle pointer to call the display function.
10. Display the rental details and total charge.
11. Release the dynamically allocated object and return to the menu.

11. Input Validation

The program checks whether the menu choice is numeric and whether it is one of the available options. It also checks the rental duration: non-numeric input is rejected, and the number of days must be greater than zero. The user is asked to enter the value again when it is invalid.

12. Sample Outputs

Test Case 1 – Car rental
Menu choice: 1
Vehicle ID: C101
Model: Swift
Rental duration: 2 days
Daily rate: Rs. 1,500
Service charge: 10%

Total rental charge: **Rs. 3,300.00**

Test Case 2 – Bike rental
Menu choice: 2
Vehicle ID: B101
Model: Activa
Rental duration: 3 days
Daily rate: Rs. 500
Total rental charge: **Rs. 1,500.00**

Test Case 3 – Invalid duration
If the user enters 0 or a negative number of days, the program displays an error and asks for the rental duration again.

Test Case 4 – Invalid menu input
If the user enters a non-numeric or unavailable menu choice, the program displays an error and asks the user to try again.

13. Testing

Test	Input	Expected result
Car rental	Car, 2 days	Total Rs. 3,300.00
Bike rental	Bike, 3 days	Total Rs. 1,500.00
Zero duration	0 days	Reject and ask again
Negative duration	-2 days	Reject and ask again
Invalid menu	Choice 9	Display invalid-choice message

14. Advantages

• Simple and user-friendly menu.
• Reuses common vehicle information through inheritance.
• Different calculations are handled by the relevant derived class.
• Encapsulation protects the vehicle data members.
• The design can be extended to support more vehicle types.

15. Limitations and Future Scope

The current project is a basic console application. It does not store customer records, maintain a vehicle database, process payments or track vehicle availability. In the future, it can be extended with customer details, booking records, file/database storage, additional vehicle categories, date-based billing and a graphical user interface.

16. Conclusion

The Vehicle Rental System demonstrates how object-oriented programming can be used to solve a simple real-world problem. The project implements classes and objects, single inheritance, constructors, encapsulation, function overriding and runtime polymorphism. It also performs vehicle-specific rental calculations and validates important user inputs. This project helped in understanding the practical use of OOPS concepts in C++.

17. References

• C++ class notes and lecture material.
• C++ language reference: https://www.cppreference.com/
• Mini-Project Assignment – OOPS/PPF.